Desarrollar software a partir de la integración de sus módulos componentes

GA8-220501096-AA1-EV01

Módulos integrados **GA8-220501096-AA1-EV02**

John Fredy Rodríguez Niño

Servicio nacional de aprendizaje

Sena

Ficha (3118493)

Análisis y desarrollo de software

Marzo 2026

Contenido

1. Introducción 3

2. Desarrollar software a partir de la integración de sus módulos componentes ga8-220501096-aa1-ev01 4

2.1 ¿Qué son las pruebas de software? 4

2.2 Tipos de pruebas que existen 4

2.3 ¿Qué pruebas usé en mi proyecto del restaurante? 5

2.4 Herramienta que usé para probar: Postman 5

2.5 Las pruebas que hice con Postman 5

2.6 Lo que aprendí en este taller 6

3.Módulos integrados ga8-220501096-aa1-ev02 6

3.1¿Qué debe hacer mi sistema? 6

3.2 ¿Qué partes del sistema ya funcionan? 7

3.3 ¿Qué partes voy a probar? 7

3.4 ¿Cómo voy a probar? 7

3.5 ¿En qué equipo y programas voy a probar? 7

3.6 ¿Cuándo voy a probar? 8

3.7 Casos de prueba (lo que voy a revisar) 8

3.8 Herramientas que voy a usar 10

3.10 Lo que aprendí haciendo este plan 12

4. Evidencias del sistema 12

5. Conclusiones 15

6. Referencias 17

7. Glosario 18

1. Introducción

Hay que asegurarse de que el programa haga bien lo que tiene que hacer. Si el programa tiene errores, los usuarios se enojan, se pierden datos y el negocio que lo usa puede tener problemas. Por eso las pruebas de software son tan importantes.

Este trabajo tiene dos partes. En la primera parte, hago un taller donde explico los tipos de pruebas que existen y cuáles usé en mi sistema del restaurante "El Colombiano". También muestro cómo usé Postman, una herramienta que permite probar los servicios web. En la segunda parte, hago un plan de pruebas, donde organizo qué voy a probar, cómo lo voy a hacer y qué voy a hacer si encuentro errores.

Todo esto lo hice pensando en el proyecto real del restaurante, que ya tiene una base de datos funcionando, servicios web para registrar clientes y listar productos, y pantallas web básicas.

2. Desarrollar software a partir de la integración de sus módulos componentes ga8-220501096-aa1-ev01

2.1 ¿Qué son las pruebas de software?

Las pruebas de software son revisiones en donde uno se fija si todo está bien, si hay errores, si falta algo, hay que probarlo para asegurarse de que funciona bien.

Probar un programa no es solo abrirlo y ver si se ve bien. Es hacer pruebas específicas para ver si cada parte hace lo que debe hacer.

2.2 Tipos de pruebas que existen

Hay varios tipos de pruebas. Cada una sirve para algo diferente.

Pruebas unitarias: Revisan partes pequeñas del programa. Por ejemplo, si tengo una función que suma los precios de un pedido, hago una prueba para asegurarme de que sume bien. Es como revisar una pieza antes de armarla con las demás.

Pruebas de integración: Revisan que dos o más partes del programa se comuniquen bien. Por ejemplo, cuando un mesero registra un cliente desde la pantalla, los datos tienen que llegar a la base de datos. La prueba de integración verifica que ese viaje de los datos se haga sin errores.

Pruebas de sistema: Acá se prueba todo el programa completo. Por ejemplo, probar que un cliente puede registrarse, luego ver el menú, luego hacer un pedido y al final pagar.

Pruebas de aceptación: Las hace el dueño del restaurante o los meseros. Ellos prueban el sistema y dicen si realmente les sirve para su trabajo. Es la prueba final antes de entregar el programa.

Pruebas funcionales: Se fijan en que cada botón y cada opción haga lo que debe hacer. Por ejemplo, que al hacer clic en "Registrar Cliente" los datos se guarden.

Pruebas no funcionales: Estas se fijan en otras cosas, como que el programa sea rápido, fácil de usar o seguro. Para mi restaurante, es importante que sea fácil de usar porque los meseros lo van a usar en tablets, celulares o pc.

2.3 ¿Qué pruebas usé en mi proyecto del restaurante?

Para mi sistema del restaurante, usé principalmente pruebas funcionales. Necesitaba verificar que las cosas importantes funcionaran bien: registrar clientes, listar productos, validar datos.

También usé pruebas de integración porque mi sistema tiene tres partes: las pantallas web, los servicios que procesan los datos y la base de datos. Tenía que asegurarme de que cuando alguien se registra, los datos lleguen bien hasta la base de datos.

Usé pruebas de validación de datos. Por ejemplo, cuando alguien escribe un correo sin el símbolo @, el sistema debe mostrar un mensaje de error. Probé eso y funcionó.

2.4 Herramienta que usé para probar: Postman

Postman es un programa que permite probar servicios web sin necesidad de tener las pantallas terminadas. Con Postman puedo enviar datos a mi sistema y ver qué responde.

Me sirvió mucho porque pude probar cada servicio por separado antes de hacer las pantallas. Así me aseguré de que todo funcionara bien desde el principio.

2.5 Las pruebas que hice con Postman

Hice varias pruebas. Las explico una por una.

Prueba 1: Conectar con la base de datos

Envié una petición al servicio que prueba la conexión con MySQL Workbench. El sistema respondió diciendo que la conexión fue exitosa. Eso me confirmó que la base de datos está funcionando y que los datos para conectarse están bien.

Prueba 2: Ver la lista de productos

Envié una petición para que me mostrara todos los productos del menú. El sistema respondió con una lista que mostraba: Bandeja Paisa, Ajiaco, Limonada de Coco, Postre de Natas, Arepa de Huevo, Tamal y Champús. Cada producto aparecía con su precio y descripción. Esto me confirmó que el menú se puede consultar bien.

Prueba 3: Registrar un cliente

Envié los datos de un cliente nuevo: nombre John, apellido Rodríguez, documento 702170702, correo jhonfre801@gmail.com, teléfono 43997232 y fecha de nacimiento 1994-09-14. El sistema respondió diciendo que el cliente se registró bien. Después revisé en la base de datos y efectivamente aparecía. Eso me confirmó que el registro funciona.

Prueba 4: Probar que el sistema detecta errores

Envié un correo sin el símbolo @ (por ejemplo, "correoinvalido"). El sistema respondió con un mensaje diciendo que el correo debe tener el símbolo @. Eso me confirmó que las validaciones funcionan y el sistema no deja pasar datos incorrectos.

2.6 Lo que aprendí en este taller

Las pruebas de software son muy importantes. No se trata solo de "probar a ver si sirve", sino de hacer pruebas específicas para cada cosa. Aprendí que hay diferentes tipos de pruebas y cada una tiene su momento.

También aprendí a usar Postman, que es una herramienta muy útil porque permite probar los servicios sin tener que esperar a que las pantallas estén listas.

3.Módulos integrados ga8-220501096-aa1-ev02

3.1 ¿Qué debe hacer mi sistema?

El sistema del restaurante tiene que hacer varias cosas importantes:

- Registrar clientes con sus datos (nombre, documento, correo, etc.)
- Mostrar el menú con los productos y sus precios
- Validar que los datos ingresados sean correctos (por ejemplo, que el correo tenga @)

- Conectarse bien con la base de datos para guardar y consultar información

3.2 ¿Qué partes del sistema ya funcionan?

Ya tengo varias cosas funcionando:

- El servicio que prueba la conexión con la base de datos
- El servicio que lista los productos del menú
- El servicio que registra clientes
- Las pantallas web: página de inicio y formulario de registro
- La base de datos con las tablas de clientes, productos, usuarios, mesas, etc.

3.3 ¿Qué partes voy a probar?

Ahora voy a probar las partes que son más importantes para que el sistema funcione bien:

- Registro de clientes: Probaré que cuando se registra un cliente con datos correctos, quede guardado.
- Listado de productos: Probaré que se muestren todos los productos con su precio.
- Validación de datos: Probaré que cuando se ingresa un dato mal, el sistema muestre un mensaje de error.
- Conexión a base de datos: Probaré que el sistema pueda conectarse sin problemas.

3.4 ¿Cómo voy a probar?

Voy a probar de dos formas:

- Con Postman: enviando peticiones a los servicios web y viendo las respuestas.
- Desde el navegador: abriendo las pantallas web y probando los formularios como si fuera un usuario.

Voy a escribir en una tabla lo que voy a probar, los pasos que voy a seguir y lo que espero que pase.

3.5 ¿En qué equipo y programas voy a probar?

Equipo:

- Mi computador portátil (Intel i5, 8GB de RAM)
- Mi teléfono para probar cómo se ve en pantalla pequeña

Programas:

- Postman para probar los servicios web
- Google Chrome para probar las pantallas web
- MySQL Workbench para revisar que los datos se guardaron bien
- Word para anotar los resultados

3.6 ¿Cuándo voy a probar?

Día	Qué voy a hacer
Día 1	Preparar todo: revisar que la base de datos esté funcionando y que los servicios estén listos
Día 2	Probar con Postman: conexión a BD, listado de productos, registro de clientes
Día 3	Probar las pantallas web: formulario de registro, que los datos se guarden bien
Día 4	Probar las validaciones: correo sin @, campos vacíos, documentos repetidos
Día 5	Corregir los errores que encuentre
Día 6	Volver a probar lo que corregí para asegurarme de que todo quedé bien

3.7 Casos de prueba (lo que voy a revisar)

Pruebas para los servicios web (con Postman):

Número	Qué voy a probar	Cómo lo voy a probar	Qué espero que pase
1	Conexión a base de datos	Enviar petición a PruebaConexionServlet	Que diga "Conexión exitosa"
2	Listar productos	Enviar petición a PruebaProductoServlet	Que muestre todos los productos del menú
3	Registrar cliente válido	Enviar datos correctos	Que diga que se registró y los datos queden guardados
4	Registrar cliente con correo malo	Enviar correo sin @	Que muestre mensaje de error

Pruebas para las pantallas web (en el navegador):

Número	Qué voy a probar	Cómo lo voy a probar	Qué espero que pase
1	Abrir la página principal	Escribir la dirección en el navegador	Que aparezca el título "Bienvenido" y el menú
2	Abrir el formulario de registro	Hacer clic en "Registrar Cliente"	Que aparezcan todos los campos
3	Registrar cliente desde la pantalla	Llenar los campos y enviar	Que salga un mensaje confirmando
4	Probar campo vacío	Dejar un campo sin llenar y enviar	Que el navegador pida llenar ese campo

3.8 Herramientas que voy a usar

Herramienta	**Para qué la voy a usar**
Postman	Probar los servicios web
Google Chrome	Probar las pantallas web
MySQL Workbench	Revisar que los datos se guardaron
Word	Anotar los resultados y errores

3.9 Cosas que pueden salir mal y qué haré si pasan

Qué puede salir mal	Qué tan probable es	Qué haré si pasa
La base de datos no responde	Baja	Revisar que MySQL esté abierto. Si no, reiniciarlo.
El servidor no despliega el programa	Media	Revisar los mensajes de error. Verificar que el programa esté bien construido.
Encuentro muchos errores	Media	Primero arreglar los errores más graves. Los pequeños los dejo para después.
Pierdo el control de lo que probé	Baja	Usar las tablas que hice para ir marcando lo que ya está probado.

3.10 Lo que aprendí haciendo este plan

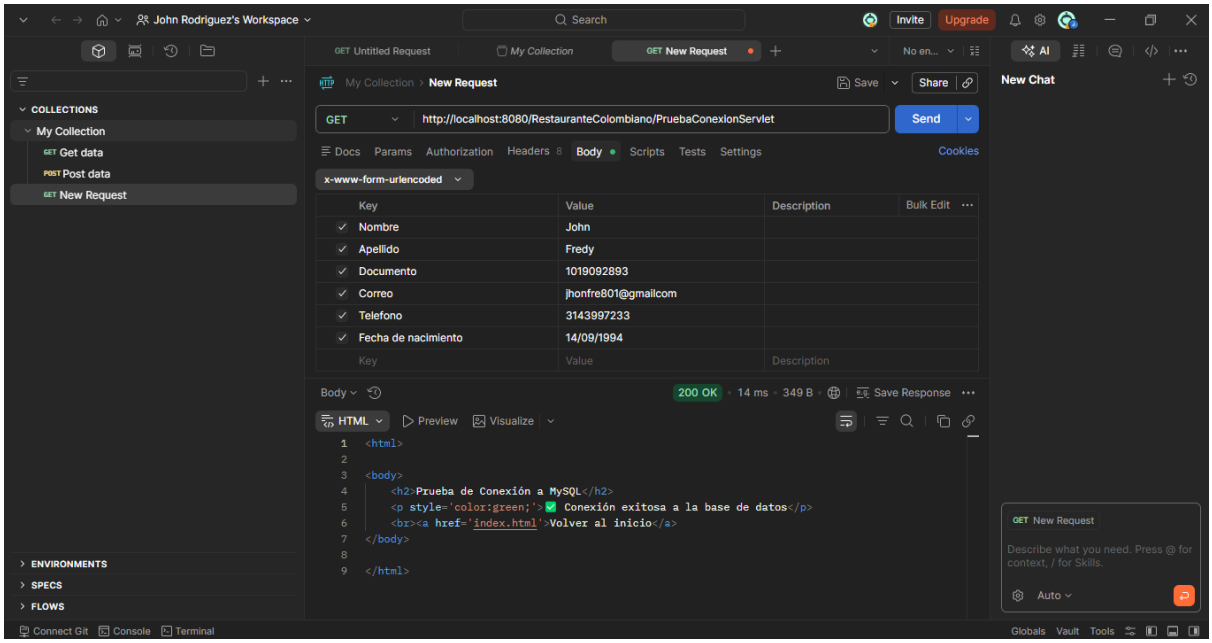
Hacer un plan de pruebas me sirvió para organizarme. Antes solo probaba sin pensar, pero ahora sé que debo definir qué probar, cómo hacerlo y en qué orden. También aprendí que es importante pensar en lo que puede salir mal y tener un plan para eso.

4. Evidencias del sistema

Aquí van las capturas de pantalla que muestran que todo funciona.

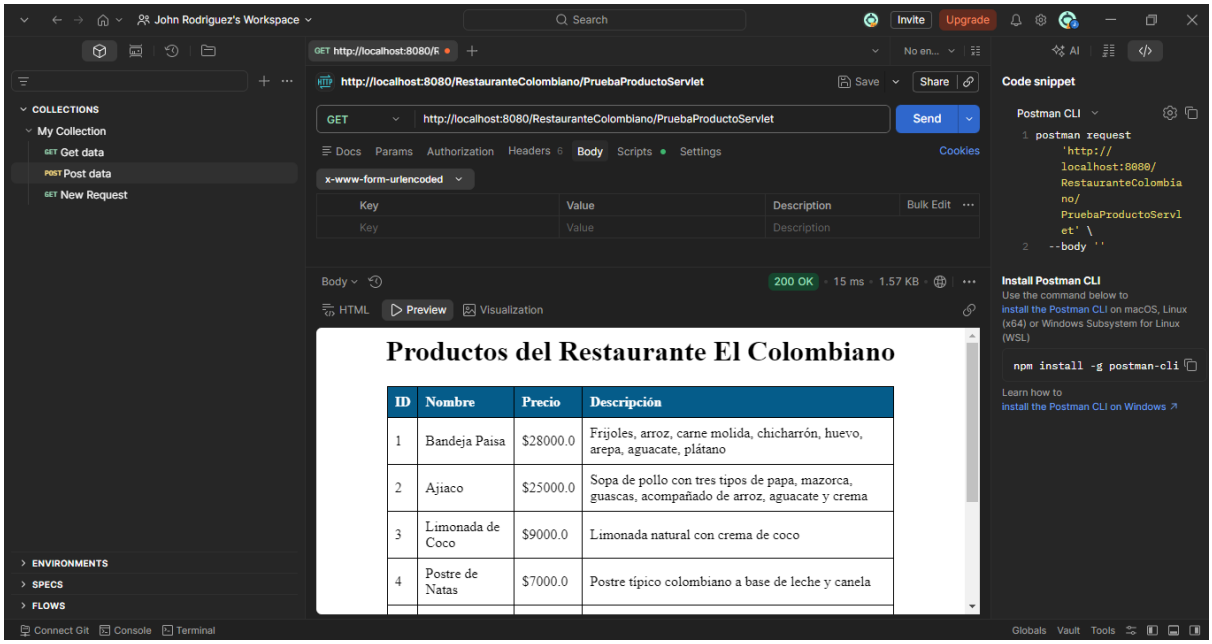
Captura 1: Prueba de conexión a la base de datos

En esta imagen se ve el resultado de probar la conexión con MySQL. El sistema respondió que la conexión fue exitosa.



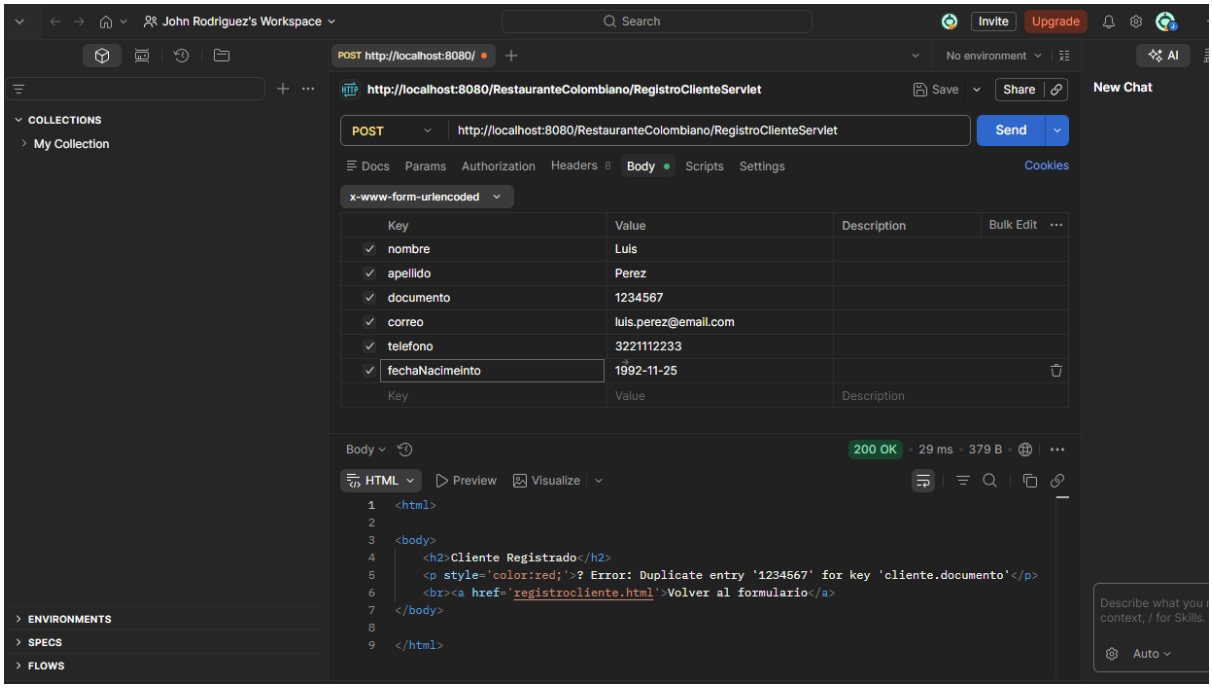
Captura 2: Listado de productos

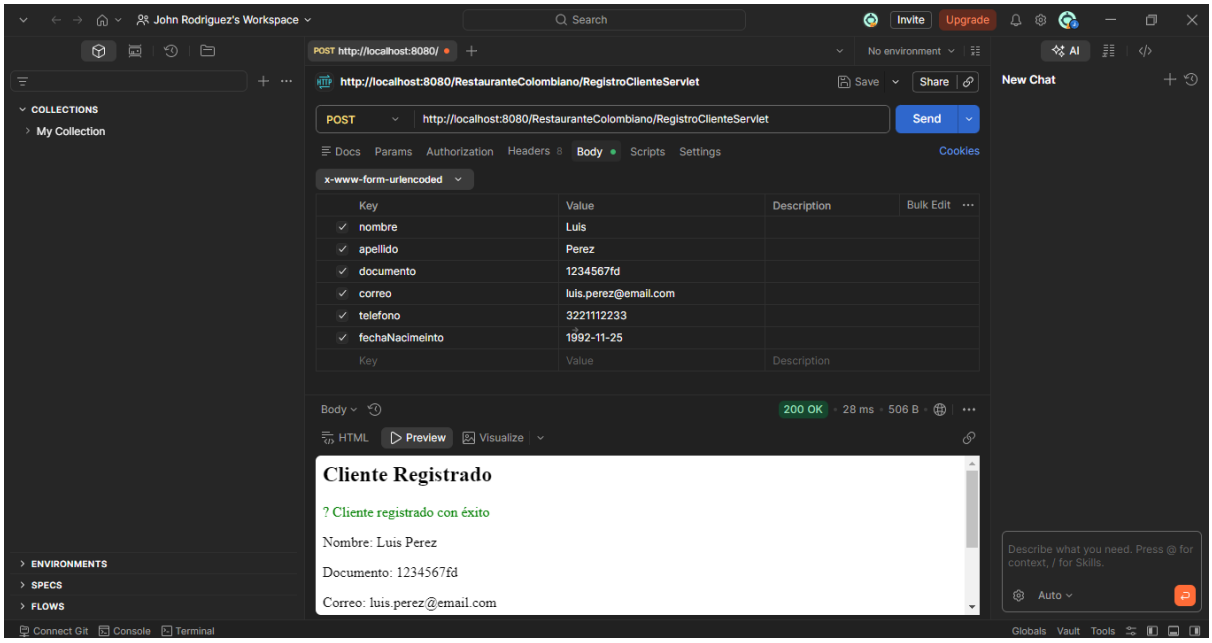
En esta imagen se ven todos los productos del menú que están en la base de datos: Bandeja Paisa, Ajiaco, Limonada de Coco, Postre de Natas, Arepa de Huevo, Tamal y Champús. Cada uno con su precio.



Captura 3: Registro de un cliente

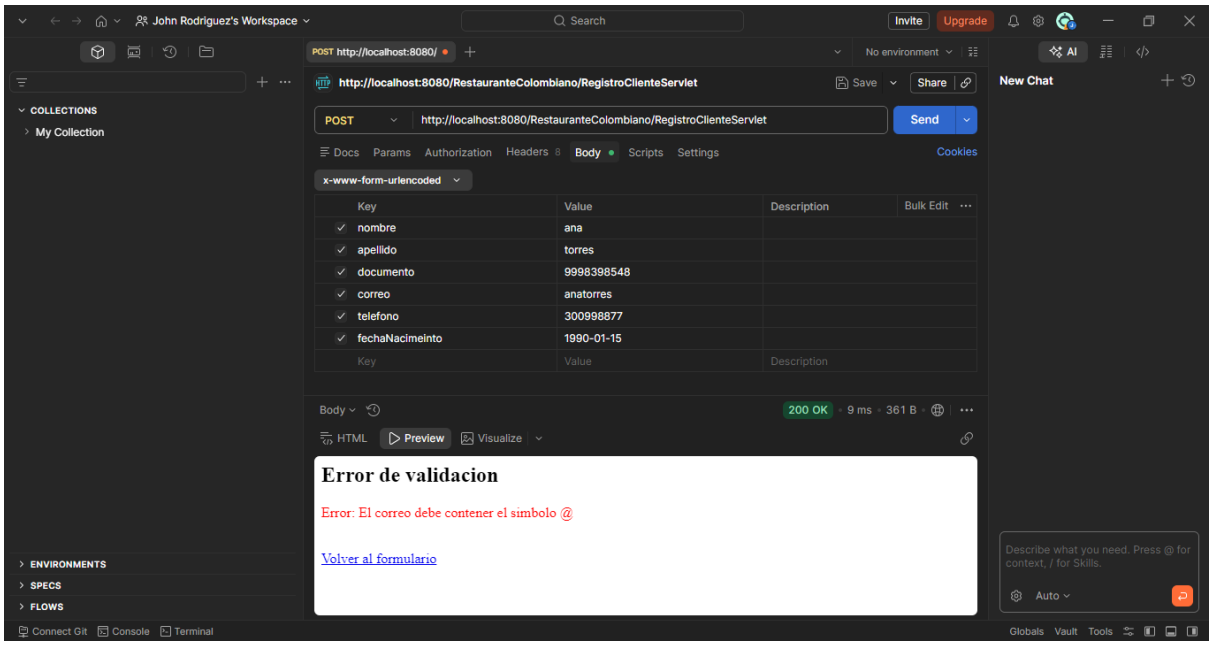
En esta imagen se ve que después de enviar los datos de un cliente nuevo, el sistema mostró un mensaje confirmando que se registró bien.

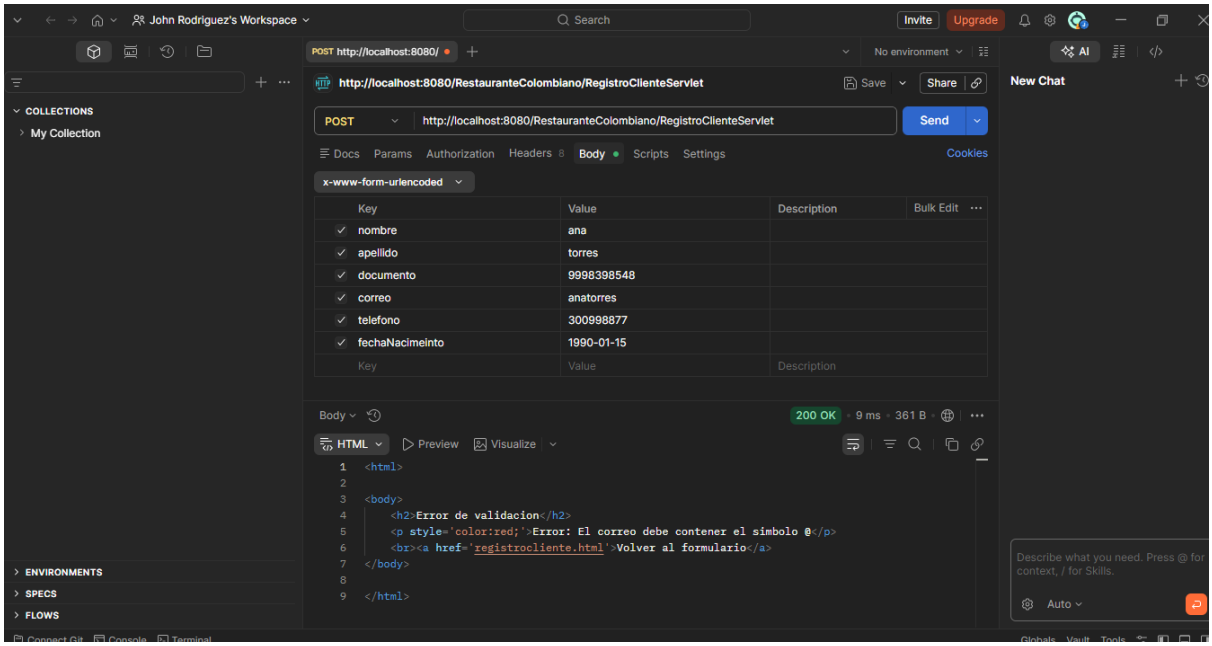




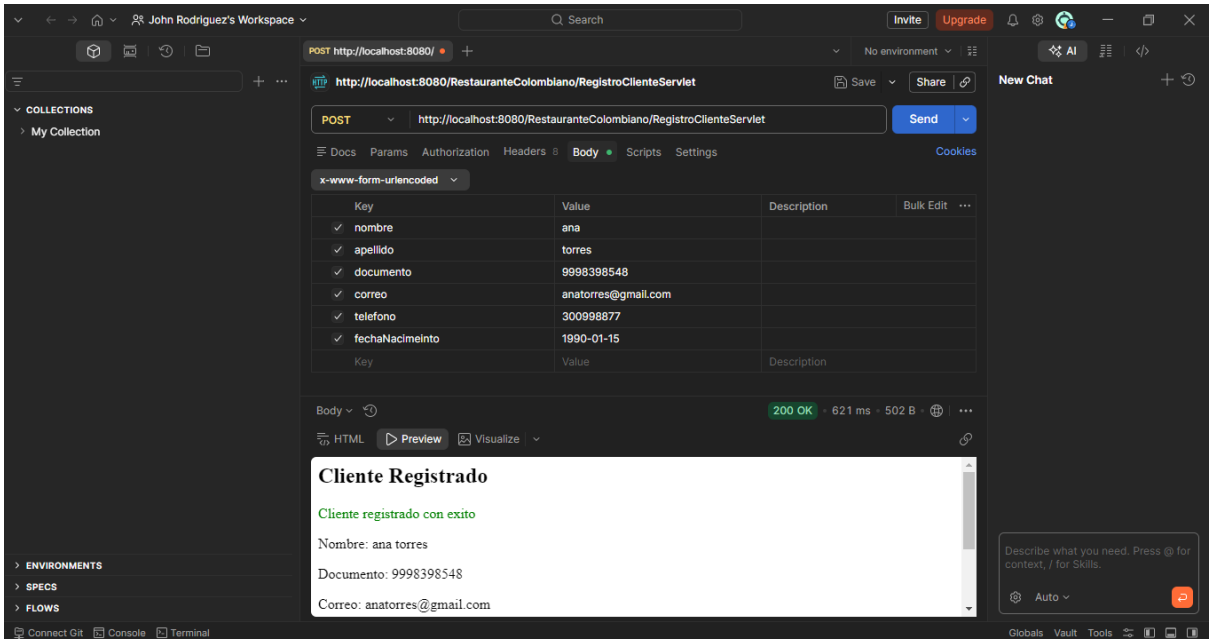
Captura 4: Error por correo inválido

En esta imagen se ve que cuando se envió un correo sin el símbolo @, el sistema mostró un mensaje de error diciendo que el correo debe tener @.




John Rodriguez's Workspace
Search
Invite
Upgrade
POST http://localhost:8080/
No environment
http://localhost:8080/RestauranteColombiano/RegistroClienteServlet
Save
Share
New Chat
COLLECTIONS
My Collection
POST
http://localhost:8080/RestauranteColombiano/RegistroClienteServlet
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
x-www-form-urlencoded
Key
Value
Description
Bulk Edit
nombre
ana
apellido
torres
documento
9998398548
correo
anatorres
telefono
300998877
fechaNacimeinto
1990-01-15
Body
200 OK
9 ms
361 B
HTML
Preview
Visualize
<html>
<body>
<h2>Error de validacion</h2>
<p style='color:red;'>Error: El correo debe contener el simbolo @</p>

<a href='registrocliente.html'>Volver al formulario</a>
</body>
</html>
ENVIRONMENTS
SPECS
FLOWS
Describe what you need. Press @ for context, / for Skills.
Auto


John Rodriguez's Workspace
Search
Invite
Upgrade
POST http://localhost:8080/
No environment
http://localhost:8080/RestauranteColombiano/RegistroClienteServlet
Save
Share
New Chat
COLLECTIONS
My Collection
POST
http://localhost:8080/RestauranteColombiano/RegistroClienteServlet
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
x-www-form-urlencoded
Key
Value
Description
Bulk Edit
nombre
ana
apellido
torres
documento
9998398548
correo
anatorres@gmail.com
telefono
300998877
fechaNacimeinto
1990-01-15
Body
200 OK
621 ms
502 B
HTML
Preview
Visualize
Cliente Registrado
Cliente registrado con exito
Nombre: ana torres
Documento: 9998398548
Correo: anatorres@gmail.com
ENVIRONMENTS
SPECS
FLOWS
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools

5. Conclusión

Hacer este trabajo me sirvió para entender que las pruebas de software son muy importantes. No es algo que se deja para el final, sino que se hace desde el principio para ir revisando que todo funcione bien.

Aprendí que hay varios tipos de pruebas: unitarias, de integración, de sistema, funcionales. Cada una sirve para algo diferente y todas son necesarias.

También aprendí a usar Postman, que es una herramienta muy útil. Con Postman pude probar mis servicios web sin necesidad de tener las pantallas terminadas. Eso me ayudó a encontrar errores temprano, cuando son más fáciles de arreglar.

Hacer el plan de pruebas me enseñó a organizarme. Ahora sé que debo definir qué voy a probar, cómo lo voy a hacer, con qué herramientas y en qué tiempo. También aprendí a pensar en los riesgos y tener un plan por si algo sale mal.

Al final, mi sistema del restaurante quedó funcionando bien. Las pruebas que hice me dieron confianza de que cuando los meseros y el dueño lo usen, no van a encontrar errores graves.

6. Referencias

Aplicación del paradigma orientado a objetos. (s.f.). Recuperado el 31 de marzo de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF28/index.html#/

Construcción de API. (s.f.). Recuperado el 31 de marzo de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF33/index.html#/

Construcción de aplicaciones con Java. (s.f.). Recuperado el 31 de marzo de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF30/index.html#/

Integración continua. (s.f.). Recuperado el 31 de marzo de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF29/index.html#/

Pruebas de software. (s.f.). Recuperado el 31 de marzo de 2026, de https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF38/index.html#/

7. Glosario

Palabra	¿Qué significa?	¿Para qué sirve?
Pruebas de software	Es el proceso de revisar un programa para encontrar errores antes de que lo usen los clientes.	Para asegurarse de que el programa funcione bien y no tenga fallos.
Pruebas unitarias	Son pruebas que revisan partes pequeñas del programa, como una función o un cálculo.	Para encontrar errores en piezas pequeñas antes de unirlas con las demás.
Pruebas de integración	Son pruebas que revisan que dos o más partes del programa se comuniquen bien entre ellas.	Para asegurarse de que los datos viajen correctamente de una parte a otra.
Pruebas de sistema	Es probar el programa completo, de principio a fin, como si fuera un usuario real.	Para verificar que todas las partes trabajen juntas sin problemas.
Pruebas de aceptación	Son las pruebas que hace el cliente o el usuario final para decidir si el programa está listo.	Para que el dueño del negocio confirme que el programa le sirve.
Pruebas funcionales	Revisan que cada botón y cada opción haga lo que debe hacer.	Para asegurarse de que las funciones del programa cumplen con lo que se pidió.
Pruebas no funcionales	Revisan cosas como la rapidez, la seguridad y lo fácil que es usar el programa.	Para que el programa no solo funcione, sino que sea rápido y cómodo.

Postman	Es una herramienta que permite probar servicios web sin necesidad de tener las pantallas.	Para enviar datos al sistema y ver qué responde, sin usar el navegador.
Servicio web	Es un programa que se comunica a través de internet y permite enviar o recibir datos.	Para que diferentes partes del sistema puedan intercambiar información.
Base de datos	Es un lugar organizado donde se guarda la información de forma permanente.	Para almacenar clientes, productos, pedidos, etc.
MySQL	Es un programa que se usa para crear y administrar bases de datos.	Para guardar la información del restaurante de manera ordenada.
Servlet	Es un pequeño programa en Java que recibe peticiones y devuelve respuestas.	Para procesar los datos que llegan desde las pantallas o desde Postman.
GET	Es un método para pedir información al servidor.	Para consultar datos, como cuando quieres ver la lista de productos.
POST	Es un método para enviar información al servidor.	Para guardar datos, como cuando registras un cliente nuevo.
Endpoint	Es la dirección web donde está disponible un servicio.	Para saber a dónde enviar los datos para que el servidor los procese.

JSON	Es un formato para organizar la información que se envía y se recibe.	Para que los datos se entiendan fácilmente entre diferentes programas.
Git	Es un programa que guarda el historial de cambios del código.	Para no perder el trabajo y poder volver a versiones anteriores si algo sale mal.
GitHub	Es una página web donde se puede guardar el código en la nube.	Para tener respaldo del proyecto y compartirlo con otros.
Validación	Es una revisión que hace el programa para asegurarse de que los datos sean correctos.	Para evitar que se guarden datos con errores, como un correo sin @.
Código de estado HTTP	Es un número que el servidor devuelve para decir cómo salió la petición.	200 significa que todo salió bien. 404 significa que no encontró lo que se pidió.